AI Image Generator

14. Change Log and Roadmap

Version 2.3
23 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Images, video and music on one GPU: Z-Image Turbo, SDXL fine-tunes, Qwen-Image, FLUX.1, LoRAs, LTX-Video, Wan 2.1, MusicGen and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and an Admin panel that manages models, LoRAs and downloads and shows live system statistics.

14. Change Log and Roadmap

14.1 Change log

Increment 1: command-line generator (21 Oct 2026)

- Created the project folder, virtual environment, and requirements.txt; installed torch 2.14.1+cu126, diffusers 0.41, transformers, accelerate.
- Wrote generate.py with prompt, negative, steps, guidance, size, seed, count and model arguments. First image: 1024x1024 in 9.7 s, peak VRAM 10.5 GB.

Increment 2: web interface

- Added app.py using Gradio 6.30 with a gallery and an info box; model loaded once at startup and kept resident. Added run-app.bat and README.

Increment 3: prompt-driven controls

- Removed style, size, steps and guidance controls. Added prompt_parser.py, the live "Will generate" line and a collapsible help table.

Increment 4: telemetry and history

- Added gpu_info.py and db.py with a 45-column SQLite table; per-image timing and GPU sampling; the History tab.

Increment 5: real 4K and 8K output

- Added upscaler.py (Real-ESRGAN x4plus, tiled, multi-pass); parser upscale words; eight upscale columns; 4K in 3.4 s, 8K in 8.5 s.

Documentation 1.0 and repository (22 Oct 2026, 09:17)

- Documentation set published as PDFs under Frazile AI ownership. Project pushed to github.com/Fzhacking/ai-image-gen (private).

Increment 6: model registry (22 Oct 2026, 10:09)

- Added models.py: SDXL base, Juggernaut XL v9, RealVisXL V5.0, DreamShaper XL v2 Turbo, Playground v2.5, Animagine XL 3.1 and FLUX.1 schnell; local checkpoints; LoRAs; compel weighting; per-model samplers and defaults; ModelManager with verified VRAM release.
- Parser: model and LoRA directives, explicit-flag tracking, apply_defaults(). App: Model dropdown with Auto, model in details and history. CLI: --model, --lora, --list-models.
- Installed peft 0.21, compel 2.5, bitsandbytes 0.50, sentencepiece, protobuf. Downloaded the five fine-tunes (32.5 GB).
- Fixed: compel padding for SDXL dual encoders; comma tidy-up; FLUX fidelity note.
- Measured: every SDXL-family model at 832x1216 in 2.0 to 10.5 s, peak 13.2 GB, swap 26 s.

Increment 7: Frazile AI branding and Upscale bar (22 Oct 2026, 10:27)

- Dark studio theme with pink-to-yellow accents, applied to both colour modes; branded header; Inter and JetBrains Mono.
- Upscale bar after generation: 2K / 4K / 8K, selected image or whole batch, records updated in place with the new db.update().

Increment 8: full width, Admin panel, Docker (22 Oct 2026, 10:50)

- Full-width layout, version pills, Clear button, "Loaded now" line.

- Admin tab with nine tiles and six sections; sysinfo.py collector on psutil and nvidia-smi; 5 s auto-refresh; optional reachability check; caching so a refresh costs 0.7 s.
- Dockerfile, docker-compose.yml and .dockerignore; app reads bind address, port and browser flag from the environment.

Increment 9: Z-Image Turbo as the default model (22 Oct 2026)

- Added the zimage family to models.py with a bf16 loader and CPU offload under 32 GB; Z-Image Turbo (Tongyi-MAI, 6B, 8 steps, Apache 2.0) is the model loaded at start-up.
- The repository ships the transformer in fp32 (23 GB); the first load converts it to bf16 and saves a 12 GB copy under models/cache/ so later loads take about about 130 s. The Qwen3 text encoder is loaded in 8-bit; everything stays on the GPU (15.8 GB resident, 18.1 GB peak).
- Measured: 1344x768 in 9.8 s and 832x1216 in 8.6 s at 8 steps; the FRAZILE AI neon-sign test rendered the text correctly.
- Auto no longer switches away from Z-Image or FLUX; style-based specialist selection applies only while an SDXL-family model is loaded.
- Fixed: CPU offload of Z-Image crashed the process on the 16 GB machine (fp32 shards plus offload exceeded system RAM); replaced by shard-by-shard loading straight onto the GPU.

Increment 10: starter LoRAs and Qwen-Image (22 Oct 2026)

- Pixel Art XL and Papercut LoRAs fetched by models.py fetch-loras; LoRAs box in the UI; with Auto, a requested LoRA switches to an SDXL model so it takes effect.
- Qwen-Image added: 20B transformer quantised to NF4 on first load and cached (11 GB), 4-bit text encoder from diffusers/qwen-image-nf4, Lightning 8-step LoRA as a permanent adapter with the exponential-shift scheduler.
- Measured: 13 to 14 s per 1 MP image at 8 steps, 18.0 GB peak, 90 s cached load. Posters, neon signs and menus render their text correctly.
- Fixed: VRAM paging on the second image (VAE tiling, cache release, expandable segments); FLUX gated-repo error now reported clearly.

Documentation 2.0 (22 Oct 2026)

- Document set rebuilt from docs-src/build_pdf.py with three new sections (Models, Admin panel, Deployment) and a model comparison figure.

Increment 11: model and LoRA management, Studio and History upgrades (22 Oct 2026)

- registry.py: models/registry.json with settings, overrides of built-in models, admin-added models and LoRA metadata (family, trigger words, default weight, source, enabled). models.py merges it into all_models(), family_models(), find_lora() and lora_info(); apply_loras() refuses LoRAs of another family.
- downloads.py: inspect_source() for Hugging Face repos and files, Civitai links, direct URLs and local paths; JobManager with a two-worker pool, byte progress, speed, ETA, resume and cancel; snapshot downloads with per-family ignore patterns; delete_model_files().
- admin_ui.py: the Admin tab rebuilt with Overview, Models, LoRAs, Add, Downloads, System, Settings and Logs; model and LoRA editors; load, unload, download, start-up, remove and delete actions; Hugging Face and Civitai tokens; clean-up; admin password lock. applog.py keeps the app's output for the Logs tab.
- Studio: quick-hint chips, live header pills (loaded model, VRAM, temperature, load, downloads), LoRA trigger-word reminders and family notes in the interpretation line, model cards, empty state, Reuse this seed and New variation; dropdowns refresh when the tab is opened so Admin additions appear without a restart.

- History: model and upscaled-only filters, a thumbnail grid backed by a 384 px JPEG cache (outputs/.thumbs/), Reuse prompt with or without seed, delete with optional file removal. db.py gained delete(), stats_by_model(), recent_counts(), lora_usage(), models_used() and fetch_filtered().
- Concurrency: ModelManager.lock plus the "gpu" and "stats" queue ids, so telemetry refreshes while an image is generating.
- Verified through the Gradio API: chips, preview, overview and system refresh, source inspection, LoRA edit, a LoRA generation that routed Auto from Z-Image to Juggernaut (6.8 s at 1024x1024, 15 steps), history with 18 thumbnails, a 46 MB LoRA (ostris/crayon_style_lora_sdxl) added through the Add flow with a background download, trigger words recorded and used in a generation, and a diffusers test repo registered, fetched and deleted again from the Models tab.

Documentation 2.1 (22 Oct 2026)

- This document set: Admin panel section rewritten around management, module reference for the six new modules, requirements FR-21 to FR-25 and NFR-11 to NFR-12, workflows for adding and managing models.

Increment 12: video and music (22 Oct 2026)

- Two new model families, video and music, with five built-in models: LTX-Video 2B, Wan 2.1 T2V 1.3B, MusicGen stereo medium and small, Stable Audio Open 1.0 (gated, untested). They appear in Admin > Models like image models, download to the Hugging Face cache (sub-folders only), and can be added from any LTX, Wan, MusicGen or Stable Audio repo through Admin > Add, which now recognises those pipeline classes and the transformers layout.
- media.py: MediaManager with loaders that stream every part straight onto the GPU in bf16 and cache a bf16 copy of the 19 to 23 GB fp32 text encoders; generate_video() and generate_music(); prepare_for() decides which of the image, video or music models must leave the card.
- Video tab (media_ui.py): six sizes, 2 to 10 s, steps, guidance, seed, start image for image-to-video, camera chips, examples; Upscale video runs Real-ESRGAN's compact video network over every frame and streams the result into a new MP4; Add soundtrack muxes a generated track under the clip.
- Music tab: MusicGen with length, guidance, temperature and seed, genre chips, examples, token-level progress; tracks saved as 16-bit WAV.
- video_io.py: ffmpeg from the imageio-ffmpeg wheel (no system install), streamed H.264 encoding, frame reading, thumbnails, audio muxing with fade-out, WAV writing and waveform pictures.
- History: kind filter and column, first-frame and waveform thumbnails, inline video and audio players, Reuse prompt returns to the tab that made the entry. Database columns kind, duration_s, fps, frames, sample_rate, soundtrack_path; counts_by_kind().
- Admin: Overview tiles for images / videos / tracks and seconds of media, Loaded now shows both managers, Load now and Unload work for video and music models, model status shows fps, limits and image-to-video support; the header pill shows both loaded models.
- Verified on the RTX 3090 through the Gradio API: see section 10.7 for the measured figures.

Increment 13: categories and batch generation (23 Oct 2026)

- A Category box in the Studio: images go to outputs/<category>/ instead of outputs/, the generations row carries the category, and the prompt is stored under the category in the new prompt_library table (created_at, category, prompt, kind, source, generation_id). History filters by category and shows it as a column; db.py categories / db.py prompts <category> print them.
- batch.py: generates a whole category through the running app's /generate endpoint (so every image has the usual telemetry), from a built-in prompt set (Nature: 50 prompts) or a text file, cycling prompts with fresh seeds; --model, --seed-start, --negative, --url, --list.
- Verified: 50 Nature images with Z-Image Turbo in one batch, saved to outputs/Nature/ and listed under the category in both tables.

- upscale_batch.py: Real-ESRGAN upscaling of a whole category (or given ids) to several targets in one run, recorded like the Studio's Upscale bar (the largest target in the upscale columns, every target under extra_json.upscales); the 50 Nature images were upscaled to 4K and 8K this way.

Documentation 2.2 (22 Oct 2026)

- This document set: new section 10 (Video and Music), later sections renumbered, module reference for media.py, media_ui.py and video_io.py, requirements FR-26 to FR-29 and NFR-13 to NFR-14, workflows 4.14 and 4.15, database columns, change log.

14.2 Roadmap

Priority	Item	Notes
High	Image-to-image and inpainting	Same pipeline family; lets a chosen image be edited rather than regenerated; the ModelManager already shares components
High	Longer and higher-resolution video	LTX-Video 13B distilled (needs the 24 GB card alone, fp8), Wan 2.2 5B at 720p, and clip extension by feeding the last frame back as a start image
Medium	Lyrics and vocals	ACE-Step or YuE for songs with vocals; both need their own packages and 8 to 16 GB of VRAM
Medium	Video LoRAs and camera-control LoRAs for LTX	The LoRA registry already knows families; a video family entry would route them
High	Character consistency with IP-Adapter or reference images	Keeps a subject's identity across prompts; the usual next step after LoRAs
High	Test the Docker image on a host with WSL 2 and Docker Desktop	Written and reviewed but unbuilt on the development machine
Medium	FLUX.1 dev and ControlNet	dev is gated and non-commercial; ControlNet gives pose and depth control
Medium	SDXL refiner or hires-fix pass	Adds generated detail at 2x before Real-ESRGAN
Medium	Export history to CSV and chart timings per model	pandas is already present
Medium	Image-to-image from a History entry	The History tab already has the record and the reuse actions
Low	Full authentication	Gradio supports basic auth for the whole app; the admin password covers destructive actions only
Low	Cloud deployment profile	The Docker image is the starting point for RunPod or a Hugging Face Space